

Final Project Report

**Made by the students: Castillo León
Carlos Ramón 21340243 Gámez Oltehua
Ángel Manuel 21340251
Gil Cota Jesús Rosario 20340171
Méndez Valenzuela, Brandon, Ismael 20340140**

**Major: Electronic Engineering Subject:
Interfaces and Test Automation.**

Teacher: Oscar Indalesio Ramírez Huerta.

Date: December 2025.

Index.

1.- Introduction.	4
2.- Description of the problem.	5
3.- General Objective.	6
4.- Specific Objectives	6
4.1.-Technical Rationale	7
5.- Design.	8
5.1 Motor Turning Circuit.....	8
5.1.2 Motor speed control circuit.....	9
5.1.3 integration of both circuits.	11
5.2- C# User Interface (UI) Design.....	12
5.3.- Class operation diagrams.....	17
5.4- Database design.	20
5.4.1 Table <i>TestResult</i>	21
5.4.2 Table <i>TestPlan</i>	23
5.4.3. Models Table.....	25
5.4.4. Table of Products.....	26
5.4.5. Table <i>TestStatus</i>	27
6.- Development.....	28
6.1.- <i>Sealevel</i>	28
6.1.1 Environment configuration.	28
6.1.2 Modbus RTU connection example	28
6.2 Instruments.	33
6.3- Development of stored procedures.	38
6.3.1- <i>AddTestPlanResult</i>	39

6.3.2- <i>Get_NextTRID</i>	39
6.3.3- <i>Get_NS_Repeated</i>	40
6.3.4- <i>Report_TestPlan</i>	41
6.3.5. <i>SP_Filter</i>	42
6.3.6- <i>Update_Producto_StatusCode</i>	44
6.4.- Development of the system	45
6.4.1- Connection to the instrument.....	45
6.4.2- Product testing.	46
6.4.3- Search	48
7.- Limitations.....	50
8.-Results.....	50
9.- Conclusion.	56
10- Bibliographies.....	56
 Fig. 5.1.2- View of the PCB design where the speed control circuit goes.	10
Fig. 5.1.3- View of the complete PCB design.	11
Fig. 5.2.1. StatusStrip del Form.	12
Fig. 5.2.3- View of the c# tail design.....	13
Fig. 5.2.3- Visualization of the tests in real time c#.	14
Fig. 5.2.4- Display of the Report's history.	15
Fig. 5.2.5- Search Window View.....	16
Fig. 5.3.1- Instrument connection.	17
Fig. 5.3.2- Operating diagram for testing a "product".	18
Fig. 5.3.3- Operation diagram for the search	19
Fig. 5.4.1- Table diagram of the database used	21



Fig. 8.1 - Register results	51
Fig. 8.2- Queue results	51
Fig. 8.3- Results of the test plans.....	52
Fig. 8.4- Results of the test plans.....	52
Fig. 8.5- Results of the test plans.....	53
Fig. 8.6- Search filtered based on serial number	54
Fig. 8.7- Search filtered based on status.	55

1.- Introduction.

This document describes the development and implementation of a communication and control system for real laboratory instruments using VISA, developed in C# language under the .NET platform. The project aims to integrate a software application with physical equipment such as a power supply and a digital multimeter, allowing its configuration and reading in an automated way.

The system is conceived as a solution aimed at automating tests and data acquisition, applying principles of object-oriented programming, use of interfaces, hardware abstraction and communication through SCPI commands. In this way, it seeks to simulate an environment close to that used in industry, where manual instrument control is replaced by programmable and repeatable systems.

2.- Description of the problem.

During the last few weeks, the Automation and Test Interfaces class learned how to use dependency injection and abstract data types (ADTs) for instrument storage, where different operations could be performed with the intention of familiarizing themselves with the logic and benefits of this design pattern technique within a simple console interface, this was with the purpose of being able to place more emphasis on what the knowledge and principles of dependency inversion (SOLID) are.

But now the knowledge acquired must be used for a system of registration and calibration of instruments, where the idea is to use what are different types of abstract data to perform more specific tasks within a visual and user-friendly interface where it is designed with fluidity and is easy for anyone to handle providing the necessary information. approaching what should be a standard in the industry and simulating what the process would be like as if it were being carried out physically.

The main problem lies in what we should consider when performing the simulation, since there are multiple considerations and requirements that we must meet for the simulation to be as close as possible to what would be a case in the industry, apart from the fact that the team will face how the design pattern logic should be used, which in this case are the interpretation of abstract data types and the injection of dependencies.

It is important to note that the circuit to implement this test interface is up to the interpretation of the students, so they must define the operations to be carried out for the tests.

3.- General Objective.

To develop an application with a graphical interface (GUI) based on Windows Forms capable of communicating with real laboratory instruments through VISA, allowing the control of a power source and the acquisition of measurements from a digital multimeter in an automated and structured way.

4.- Specific Objectives.

- Design and build a physical circuit with fixed measurement points, of different types of signal or magnitude.
- Define at least two different models, each with different acceptance parameters for the tests to be performed.
- Allow controlled variation of parameters in the circuit in order to generate test conditions that include PASS and FAIL cases.
- You must record the Test plan for each model.
- Previously register the serial numbers to be tested, indicating the model to which each one belongs.
- Development of a C# application using class libraries and a Windows Forms interface:
 - Obtain the TestPlan corresponding to the model of each serial number to be tested from the DB.
 - Control instruments by GPIB and/or Serial.

- o Sample 30 measurements per test, calculate the average to up to 4 decimal places, and add the unit of measurement.
 - o Evaluate the measurement result against the defined limits and determine the PASS/FAIL status of each test.
 - o Record the results in the database.
- Implement logic so that if all tests for a serial number are PASS, the product is marked as PASS; in case of any failure, the tests are stopped and the serial number is marked as FAIL, but all tests performed including the failed one are recorded
- Include a report menu in the system

4.1.-Technical Basis.

The project was developed under the conditions of graphical user interfaces (GUI) in C# with Windows Forms, using:

- **Language:** C# (.NET Framework 8.0)
- **Environment:** Visual Studio 2026
- **Paradigm:** Object-Oriented Programming (OOP)
- **Comunicación:** VISA (Ivi.Visa / NationalInstruments.Visa)
- Communication with the ssealevel via modbus
- **Protocolo:** SCPI (Standard Commands for Programmable Instruments)
- **Application Type:** Class Library, Console, and Windows Forms App.

A main "Proyecto_Final_Interfaces" form was designed with a "TabControl" divided into five operational sections: *Register*, *Queue*, *Test panel*, *History* and *search*. Each module meets a specific and independent functional objective, contributing to the complete flow of the system. The system is decoupled by injection of dependencies in which the service (or controller) depends on interfaces, *I_Instruments*, *I_Source* and *I_Multimeter* are examples of some where the next chapter will talk in more detail.

5.- Design.

5.1 Motor rotation circuit.

This circuit uses a DPDT (*Double pole double throw*) relay to reverse the polarity applied to the DC motor. When the polarity changes, the direction of rotation is reversed, The relay drive is carried out by a *Sealevel* activated by a C# interface and a 1N4007 diode, which protects the circuit against voltage spikes generated by the relay coil.

Main Components.

- Motor DC 12V.
- Relé DPDT.
- Diode 1N4007.
- *Sealevel*.
- 12 volt power supply.
- 24 volt power supply (to power the *Sealevel*).

How it works

Activating an assigned relay at the *sealevel* , in this case the 2, activates a section of the motor to start rotating clockwise, switching its contacts and applying a specific polarity to the motor.

By activating relay 3 at the *sealevel* , the counterclockwise direction is activated, the relay reverses the internal connections, changing the polarity of the motor.

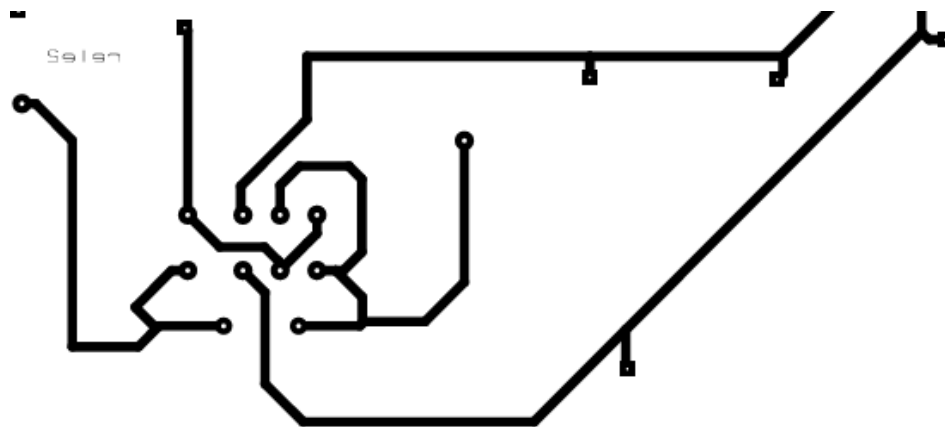


Fig. 5.1- View of the PCB design where the rotation circuit goes.

5.1.2 Motor speed control circuit.

The speed control circuit allows the DC motor revolutions to be regulated by varying the effective voltage applied. This is achieved by means of a potentiometer and which acts as a power regulator.

Components.

- 50k power meter.

- Resistance.
- Capacitors.
- Shielding diode.
- 12 Vdc power supply.

Operation.

The circuit is activated when the *sealevel* activates the relay 1,5,8 separating it from the rotation circuit to avoid disturbances in the speed, this can vary its speed by means of the potentiometer varying the current and voltage that reach the motor.

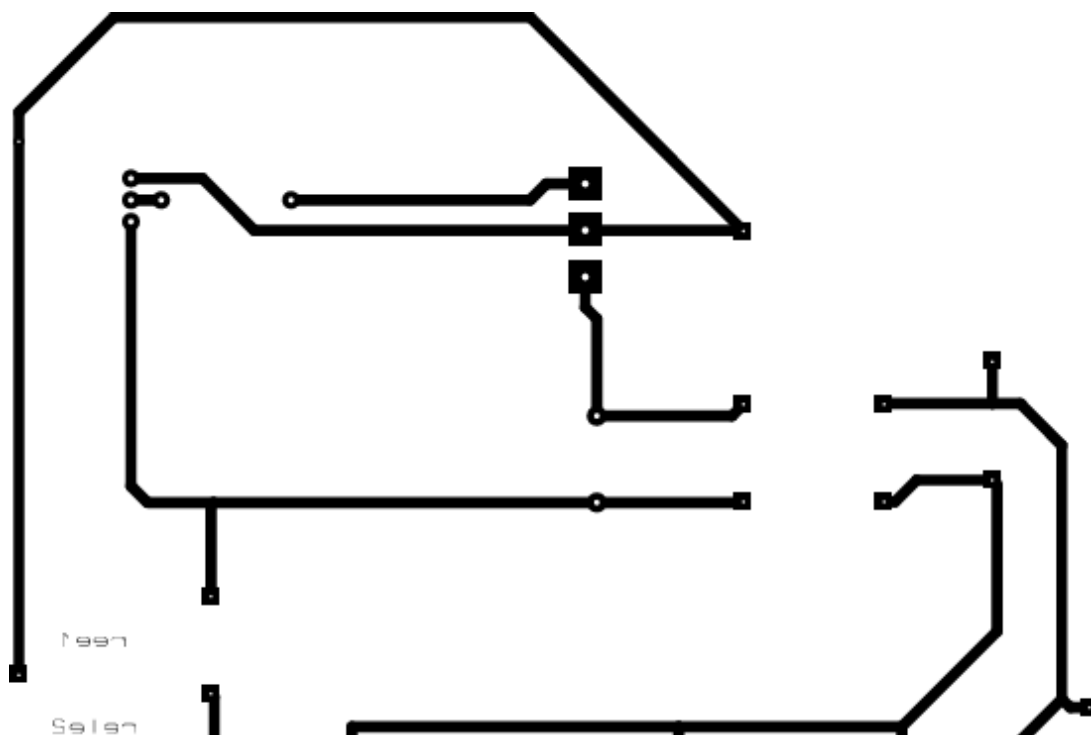


Fig. 5.1.2- View of the PCB design where the speed control circuit goes.

5.1.3 integration of both circuits.

By integrating both circuits, the control depends entirely on the *sealevel*, which will drive each part of the circuit separately.

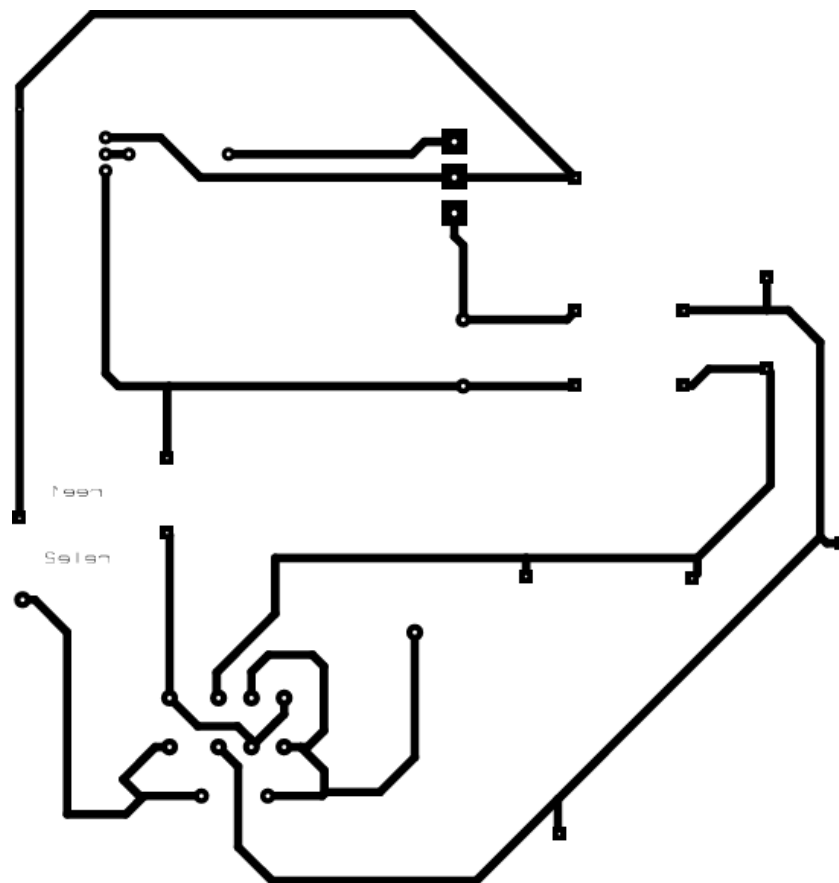


Fig. 5.1.3- View of the complete PCB design.

5.2- C# User Interface (UI) Design.

As mentioned in chapter 4.1, this program makes use of 5 windows intended to be a graphical interface that connects with instruments, *Sealevel*, and SQL database for data visualization. The *Form* is also made up of a *StatusStrip* where different Labels will be displayed to indicate a reference to crucial elements of the program, these being the queue (FIFO), notifications or recent messages and the current weather. The following image indicates the elements mentioned. It is important to note that no matter which window the user chooses to look at, the *StatusStrip* will always be visible.

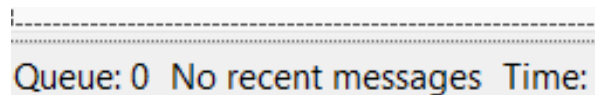


Fig. 5.2.1. *StatusStrip* del *Form*.

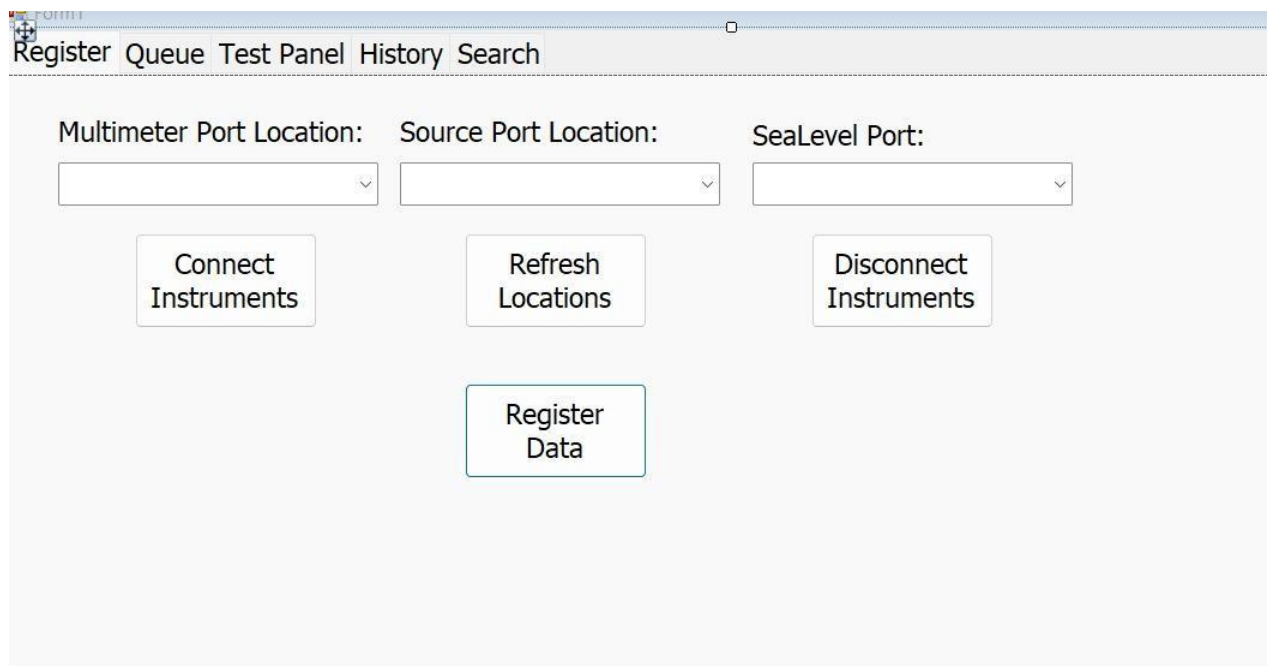


Fig. 5.2.2- Design view of recording data.

As shown in Figure 5.2.2 which corresponds to the System Instrument Registration window or *Register*. In the section the user can select the communication ports for each measuring equipment, from the same visa the main connection actions are controlled, allowing the instruments to be connected, to update or refresh the available ports and to disconnect the equipment. The registration button must load all the *TestPlans* to be evaluated by the instruments where they can be seen below.

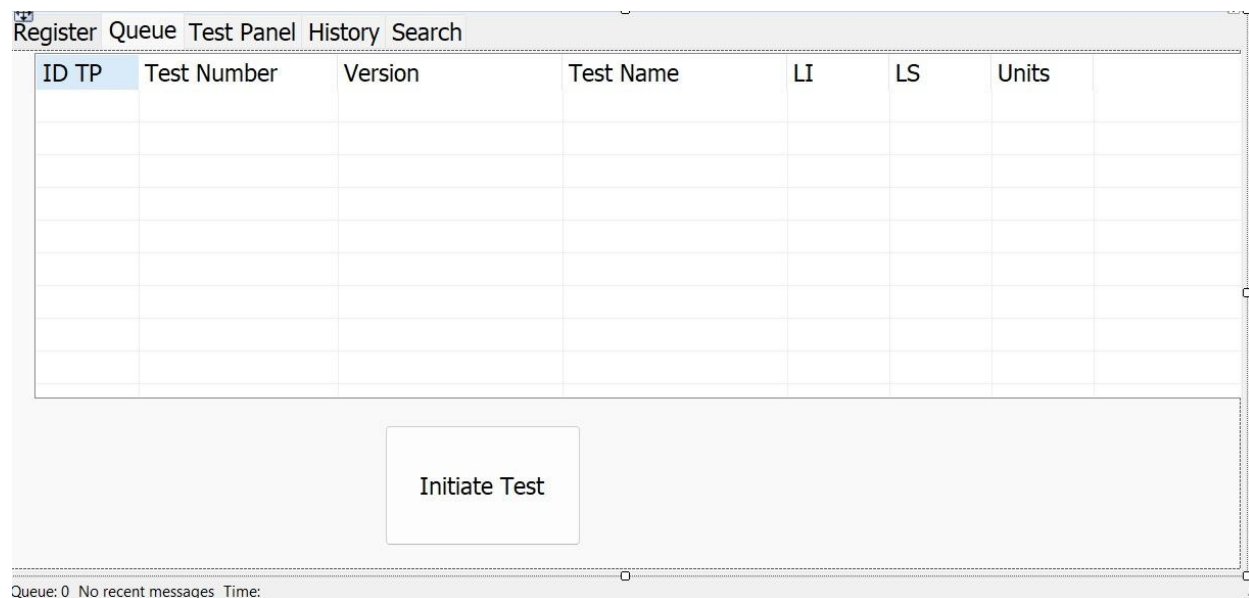


Fig. 5.2.3- View of the c# tail design.

The figure shows the interface of the test selection and initiation module within the application, the main area of the interface is made up of a data table (which is a *ListView* object), in which the available plans or tests are listed. The table includes fields such as *TP ID*, *Test Number*, *Version*, *Test Name*, *LI*, *LS* and *Units*, which allow you to identify each test, its version and the lower and upper limits of operation, as well as the corresponding units of measurement.

At the bottom is the *Initiate Test* button, whose function is to start the selected test within the *ListView*, this test is designed to perform the operations of all the *TestPlans* of the registered product.

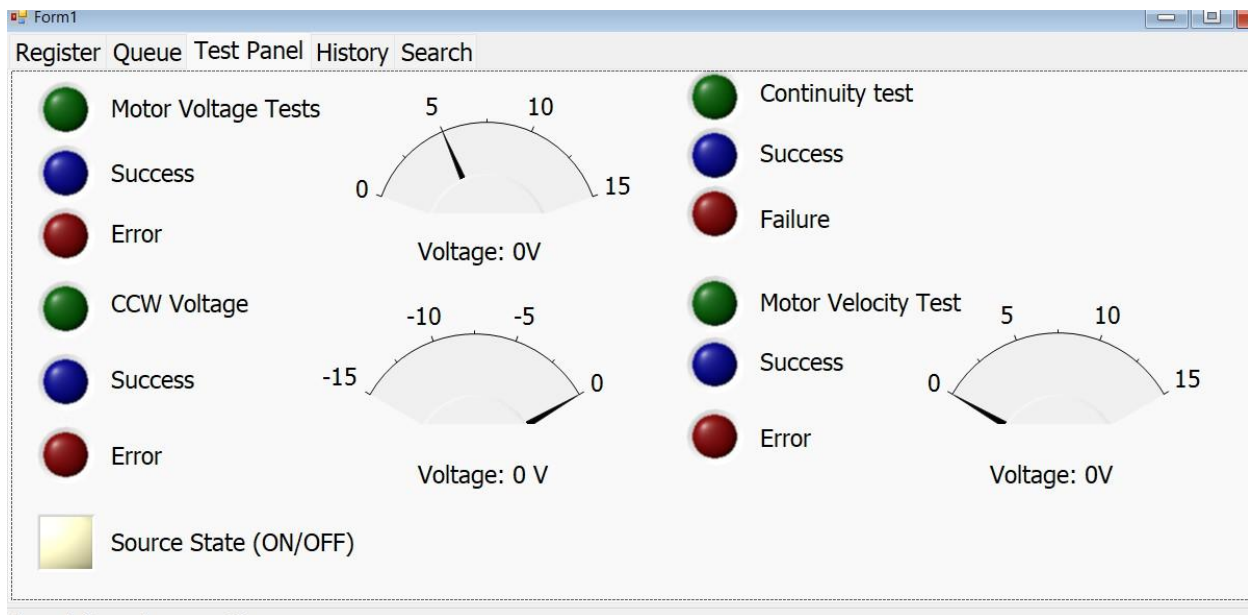


Fig. 5.2.3- Visualization of the tests in real time c#.

The figure shows the graphical interface of an electrical test system used for voltage measurement in a motor and electrical continuity verification, visual indicators are used to show the status of the tests, where the green LED indicates active test, blue success and red error. In the central part there are three analog measurements (*meter*): one for the voltage of the motor in the normal direction (0 to 15V), for the voltage in the opposite direction (*CCW*) with a negative scale and the last one is for a speed test different from the first test (with the same scale from 0 to 15V). Both meters indicate 0V, which indicates that the motor is not energized at the time of initializing the application and also when a new test is desired for a product of different serial number. The continuity testing section is

similar to that of voltages, but unlike there is no scale (meter) for analog reading, so it allows to verify the continuity of the circuit, also using status indicators by chlores. Finally, the source status LED shows its status.

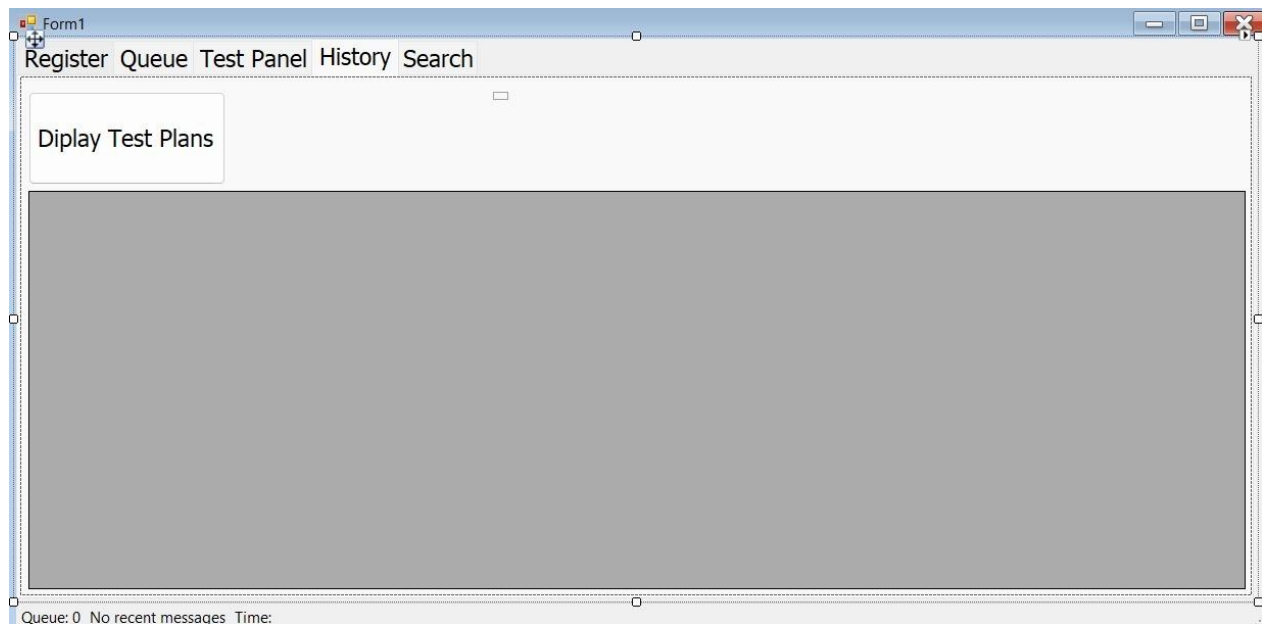
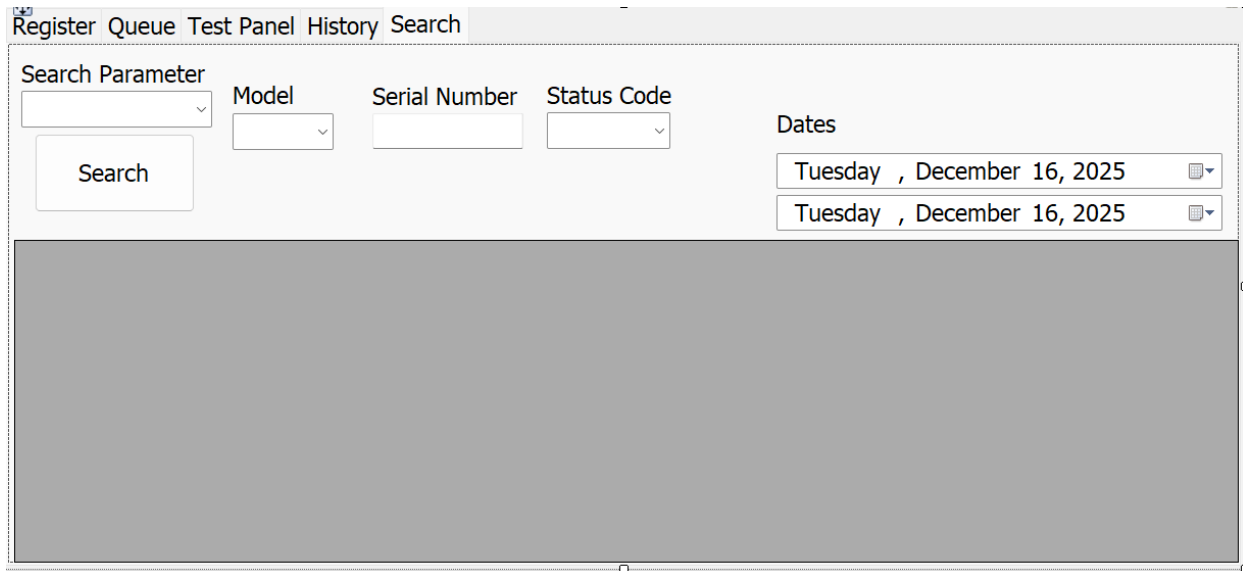


Fig. 5.2.4- Display of the Report's history.

Figure 5.2.4 presents the results of the tests involved in the project, which allow the history to be consulted. The button (*Display Test Plans*), intended to display or load the *TestResults*. Most of the interface is occupied by a central work area, intended for the visualization of information using a DGV (*DataGridView*).



The screenshot shows a software window with a menu bar containing 'Register', 'Queue', 'Test Panel', 'History', and 'Search'. The 'Search' menu is active, displaying a search form. The form includes a 'Search Parameter' dropdown menu, a 'Search' button, and four input fields: 'Model', 'Serial Number', 'Status Code', and 'Dates'. The 'Dates' field is expanded, showing two date pickers both set to 'Tuesday , December 16, 2025'. Below the input fields is a large, empty gray rectangular area, likely a placeholder for search results.

Fig. 5.2.5- View of the search window.

In this section the components are shown to start the search for a specific item, to start all the *ComboBox* present will contain all the possible items that can be found in the respective field, for example; model would contain "A" or "B" which are the 2 different versions of the product. The Search Parameter *ComboBox* will indicate which of the different fields present will be taken into account to filter out unwanted items.

The only 2 different elements for queries are the *serial number TextBox* , with the intention that the operator will be able to query at will and the 2 *DateTimePickers* to query the range of dates to be searched.

5.3.- Operating diagrams and classes.

From now on, we will talk about the architecture of the program, what it must comply with and what the logic will be for the development of this project.

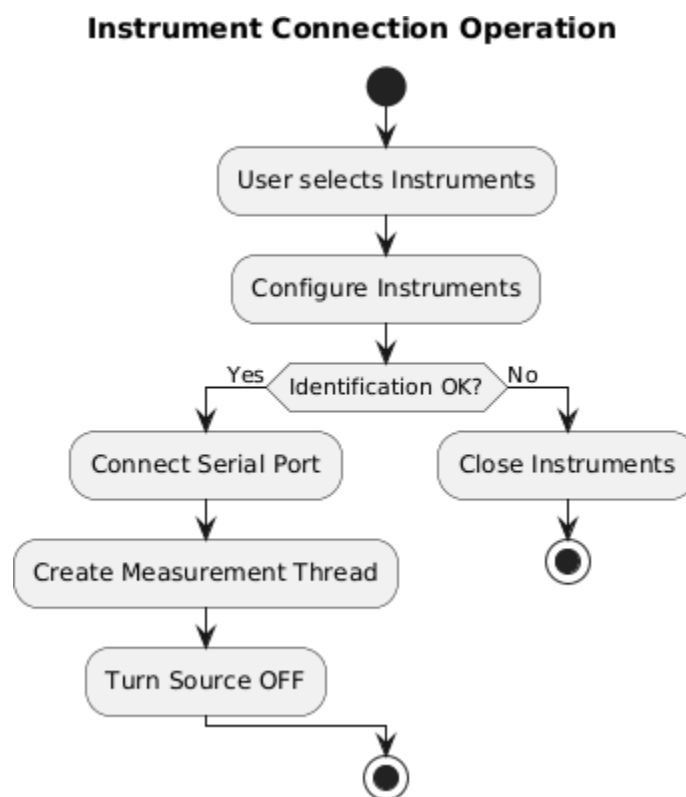
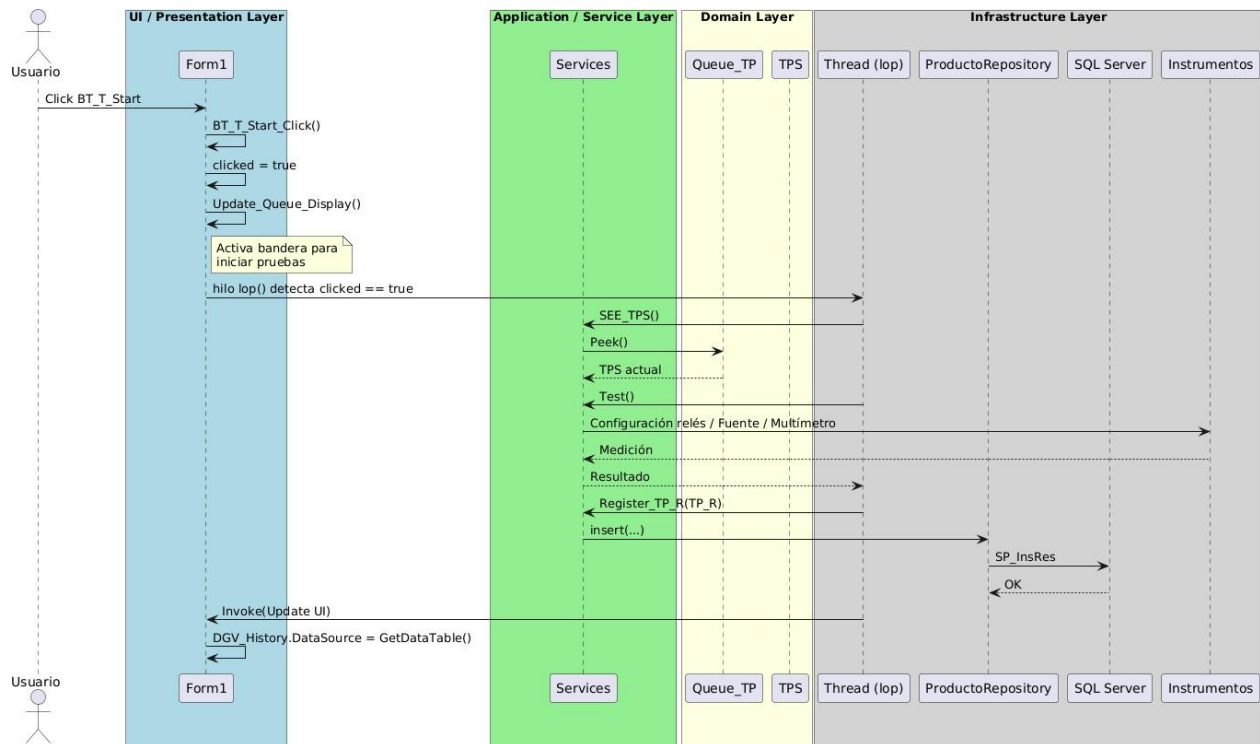


Fig. 5.3.1- Instrument connection.

This diagram shows a system that, when connecting the instruments, the program asks to identify them so that, when connecting, they can make sure that they are the right ones autonomously.



This is the most important section of the program, allowing you to communicate with different parts of the program to perform the tests and evaluate them where you can see the results in the *Test Panel* section of the interface and then register them in the database.

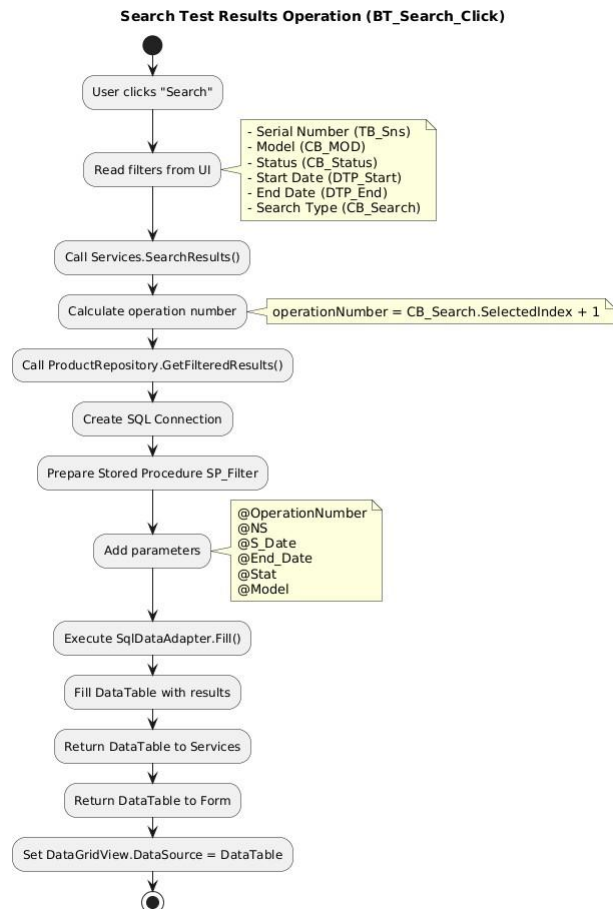


Fig. 5.3.3- Operation diagram for the search.

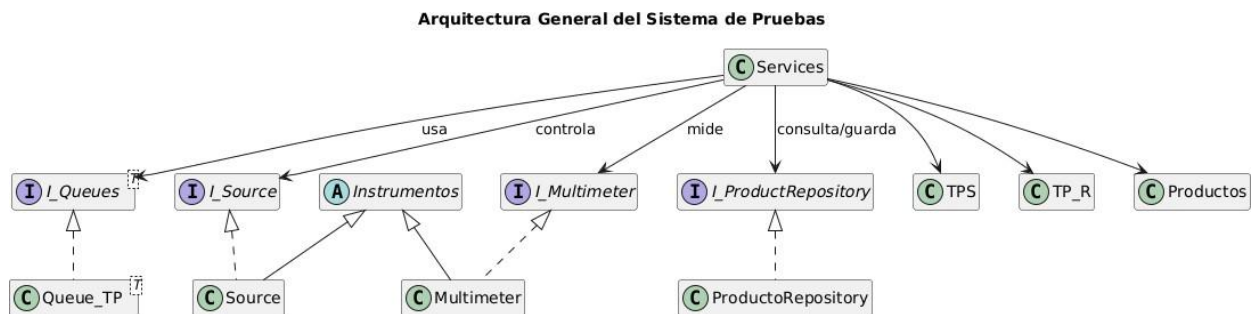


Fig. 5.3.4- Simplified class diagram

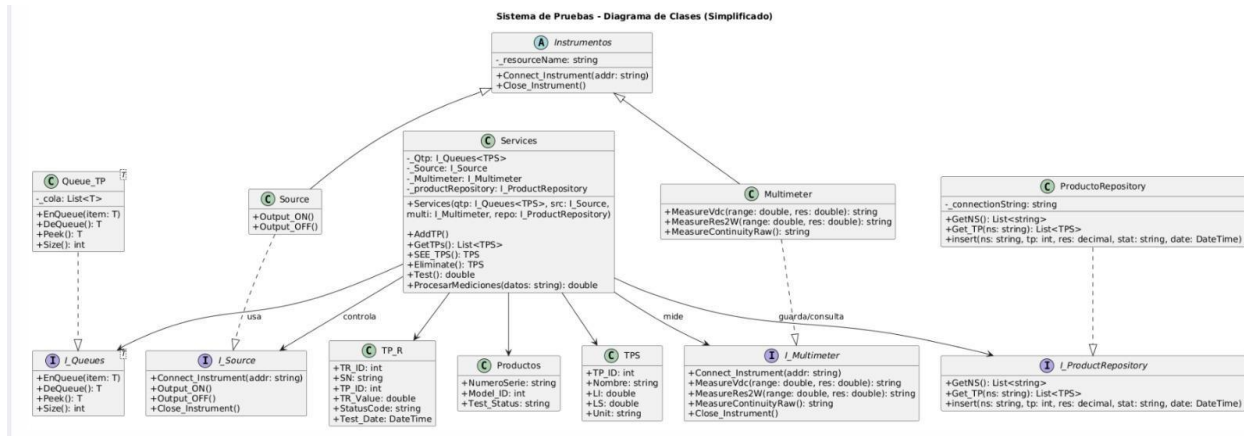


Fig. 5.3.5- Developed class diagram

In Figure 5.3.5 the system design is based on a modular structure, using abstract classes, interfaces and concrete classes to ensure flexibility, scalability and ease of maintenance.

Instruments encapsulate connection and operation details, the test queue protects its internal structure, domain classes safeguard test, result, and product information, and the repository hides database access logic.

5.4- Database design.

The database was provided by the teacher with the intention of providing a foundation for students to focus on developing the programming of the interface, controller, and other systems necessary for the development of the project. The structure of the database will be explained below.

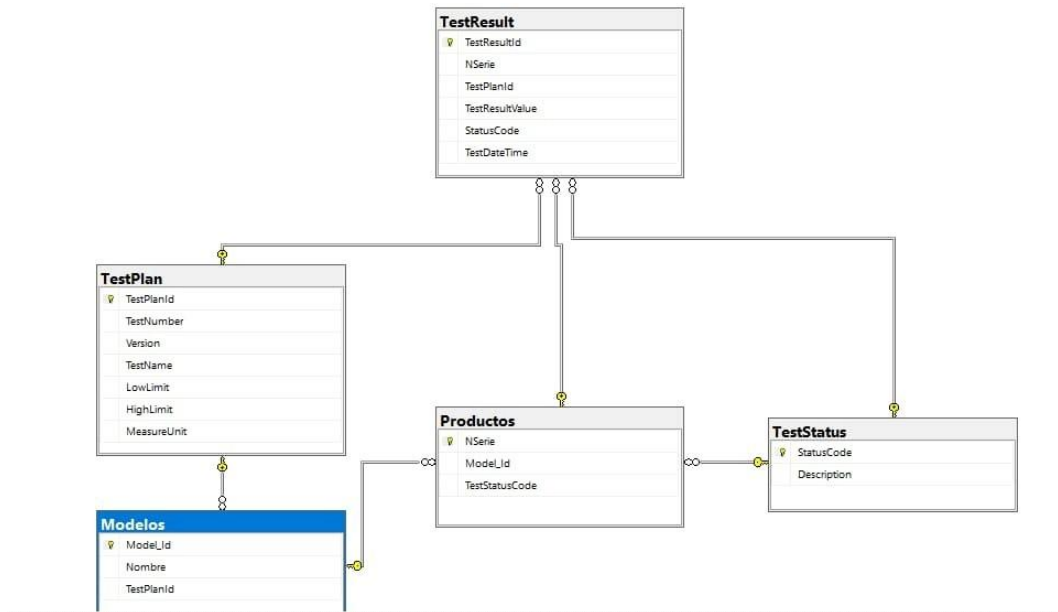


Fig. 5.4.1- Table diagram of the database used.

5.4.1 TestResult table .

❖ *TestResultId.*

- What it's for: Uniquely identifies each test result.
- Where it's used:
 - Automatically generated when inserting a result.
 - Consulted in *Get_NextTRID* for reference of the following record.
 - Used in historical reports and queries.

❖ *NSerie.*

- What it's for: Relate the result to a specific product.
- Where it's used:

- Insertion of test results.
- Consultations in *TestResult_History*.
- Direct relationship to the Products table.
- ❖ *TestPlanId*.
 - What it's for: Indicates which test plan was applied.
 - Where it's used:
 - Insertion of results.
 - Reports in *Report_TestPlan*.
 - Relationship to the *TestPlan* table.
- ❖ *TestResultValue*.
 - What it's for: Stores the measured value during the test.
 - Where it's used:
 - Record of evidence.
 - Visualization in history and reports.
 - Evaluation against defined limits.
- ❖ *StatusCode*.
 - What it is for: Define whether the test was approved or rejected.
 - Where it's used:
 - Insertion of results.

- Relationship to *TestStatus*.
- Consultations in *TestResult_History*.

❖ *TestDateTime*.

- What it's for: Records when the test was run.
- Where it's used:
 - Ordering of results in *TestResult_History*.
 - Chronological reports.

5.4.2 TestPlan Table .

❖ *TestPlanId*

- What it's for: Unique identifier of the test plan.
- Where it's used:
 - Relationship with Models.
 - Insertion and querying of results (*TestResult*).
 - SP: *AddTestPlan*, *UpdateTestPlan*, *Report_TestPlan*.

❖ *TestNumber*

- What it is for: Number that identifies the test.
- Where it's used:
 - Definition of the plan.
 - Visualization in reports (*Report_TestPlan*).



❖ *Version*

- What it's for: Test plan versioning.
- Where it's used:
 - Update in *UpdateTestPlan*.
 - Test reports.

❖ *TestName*.

- What it's for: Describes the type of test performed.
- Where it's used:
 - Reports.
 - Test history.
 - Plan administration.

❖ *LowLimit / HighLimit*.

- What it is for: Define the acceptable ranges of the result.
- Where it's used:
 - Logical evaluation of results.
 - Comparative reports.

❖ *MeasureUnit*.

- What it is for: Indicates the unit of measurement.
- Where it's used:

- Interpretation of the result.
- Reportes (*Report_TestPlan*).

5.4.3. Models Table.

❖ *Model_Id.*

- What it is for: Identify the model of the product.
- Where it's used:
 - Relationship with Products.
 - Update and registration (*AddModel*, *UpdateModelo*).

❖ *Name.*

- What it's for: Technical name of the model.
- Where it's used:
 - Reports (*TestResult_History*).
 - In-app display.

❖ *TestPlanId.*

- What it's for: Define which tests correspond to the model.
- Where it's used:
 - When registering products.
 - Automatic relationship with results.

5.4.4. Table of Products.

❖ *NSerie.*

- What it's for: Unique product identifier.
- Where it's used:
 - Recording of results (TestResult).
 - Validation (Get_NS_Repeated).
 - History and reports.

❖ *Model_Id.*

- What it is for: Indicate which model the product belongs to.
- Where it's used:
 - Consultation of the applicable test plan.
 - Relationship with Models.

❖ TestStatusCode

- What it's for: Summarize the final state of the product.
- Where it's used:
 - It must be updated at the end of the test for a model.
 - Quick status queries.



5.4.5. TestStatus table.

❖ *StatusCode.*

- What it's for: Trial status code.
- Where it's used:
 - A *TestResult*.
 - In Products.
 - Relationship with results and final status.

❖ *Description.*

- What it is for: Explanation of the state.
- Where it's used:
 - Display for the user in this same table.

6.- Development.

In this section, all the considerations or work carried out during the realization of this program will be described with the intention of explaining why or how it was that we carried out the project in such a way to meet the objectives established at the beginning of the document.

6.1.- *Sealevel*.

Sealevel devices allow you to control digital and analog inputs and outputs using standard protocols such as Modbus. With Modbus4 for C#, it is possible to send read/write commands to registers of the *Sealevel* Modbus RTU (Serial Port) or Modbus TCP.

6.1.1 Environment configuration.

- Instalar Modbus4: *Dotnet add package Modbus4*.
- Configure the serial port.
- Baud rate: according to *SeaLevel specifications*.
- Parity: *None* or *Even* depending on the model
- Use a library that is able to distinguish the port from the *SeaLevel*, in this case *System.IO.Ports*.

6.1.2 Example of a Modbus RTU connection.

Here you can witness fragments essential for the use of the *SeaLevel*. In the following excerpt you can see the bookcases used.

```
using System;  
using System.IO.Ports;  
using Modbus.Device;
```

This snippet demonstrates all the essential elements when creating the serial port configuration.

```
SerialPort Serial Port Settings = new  
SerialPort(  
    "COM3",        // Puerto COM  
    9600,           Baud rate Parity.None,  
                   No parity  
    8,             Data Bits  
    StopBits.One   Stop Bit  
);
```

Opening of the port port.

```
Open();
```

Creating the Modbus RTU Master

```
var master = ModbusSerialMaster.CreateRtu(puerto);
```

// ID del esclavo Modbus

```
byte slaveID = 1;
```

Reading a Digital Input (Input 0)

```
bool [] inputs = master. ReadInputs(slaveID, 0, 1);
```

```
Console.WriteLine("Estado de la entrada 0: " + inputs [0]);
```

Writing a Digital Output (Coil 5)

```
master. WriteSingleCoil(slaveID, 5, true);  
Console.WriteLine("Salida 5 activada");  
  
// Cierre del puerto serial  
puerto. Close();  
}  
}
```

This is a basic example of the different possible operations using the modbus library.

RELAY FUNCTIONS

```
private void ActivateRelay(int number)  
{  
    int real = numero - 1; master.  
    WriteSingleCoil(slaveId, (ushort)real, true);  
}  
  
private void DisableRelay(int number)  
{  
    int real = numero - 1; master.  
    WriteSingleCoil(slaveId, (ushort)real, false);  
}
```



The previous functions are to activate the relay or deactivate, it has a subtracter since the program looks at it from 1 to 8, but the sealevel perceives it from 0 to 7 as bist.

```
testSpeedActive = true;

bt_velocidad. Enabled = false;
bt_giro. Enabled = false;

ActivarRele(5);
ActivarRele(8);
ActivarRele(1);

await Task. Delay (10000); // 10 s

DeactivateRelay
(1);
DeactivateRelay
(5);
DeactivateRelay
(8);

testSpeedActive = false;
```

Sequence of activation of the circuit for the speed test, closes the relays 1,5,8 to be able to separate it from the rotation circuit, removing the energy and activating only the speed circuit which is varied by means of a potentiometer simulating different models.



```
testActiveSpin = true;

bt_velocidad.Enabled = false; bt_giro.
Enabled = false;

ActivateRele(2);
await Task.Delay(7000);
DesactivarRele(2);

// ---- Espera 1 s ----
await Task.Delay(1000);

ActivateRele(3);
await Task.Delay(9000);
DesactivarRele(3);

testGiroActiva = false;

bt_velocidad.Enabled = true; bt_giro.
Enabled = true;
```

Rotation sequence of the motor being controlled with the sealevel, activating the rotation clockwise with relay 2 and counterclockwise with relay 3.

6.2 Instruments.

In this part of the code, we will define the service layer responsible for communicating a C# application with real laboratory instruments (power supply and multimeter) using VISA (Ivi.Visa / NationalInstruments.Visa) and an interface-oriented class hierarchy (I_Instruments, I_Source, I_Multimeter). Its main purpose is to offer a unique and reusable abstraction to connect, configure and read physical instruments from the software, saving the rest of the project from having to worry about low-level details such as VISA sessions, timeouts or manual handling of SCPI commands.

```
namespace Service
{
    public abstract class Instrumentos : I_Instruments, IDisposable
    {
        private string _resourceName;
        private ResourceManager rm;
        protected MessageBasedSession mbSession;
```

The abstract Instruments class acts as the basis for all VISA devices. Centralize connection, SCPI communication, and resource release. Its purpose is to avoid duplicate code and offer a uniform structure for any equipment in the laboratory, while maintaining a professional and scalable design.

```
public void Close_Instrument()
{
    if (mbSession != null)
```

```
{  
    mbSession.Dispose();  
    mbSession = null;  
}  
  
if (rm != null)  
{  
    rm.Dispose();  
    rm = null;  
}  
}
```

Method in charge of closing the connection and releasing the occupied VISA resources. Ensures that the session is not open and that there are no communication blocks. This process is critical in automated measurement applications.

```
public void Dispose()  
{  
    Close_Instrument();  
}
```

Implementation of the IDisposable pattern to allow a secure closure of the instrument.

```
public void Connect_Instrument(string resourceName)  
{  
    try
```

```
{  
    rm = new ResourceManager();  
    mbSession = (MessageBasedSession)rm.Open(resourceName);  
    mbSession.ReadStatusByte();  
    mbSession.TimeoutMilliseconds = 5000;  
}  
catch (Exception ex)  
    {Console.WriteLine("Error connecting to instrument: " + ex.Message); }  
}
```

This is a method that establishes the connection to the instrument using its *ResourceName*. Create the *ResourceManager*, log in, and set the *timeout*. This method is the basis for initiating any measurement process.

```
public void Write(string command)  
{  
    if (mbSession != null)  
    {  
        mbSession.RawIO.Write(command);  
    }  
    else  
    {  
        throw new InvalidOperationException("Instrument is not connected.");  
    }  
}
```

Sends an SCPI command to the instrument. It is the fundamental control operation, allowing you to change settings or activate internal functions of the equipment.

```
public string Read()  
{  
    if (mbSession != null)  
    {
```

```
        return mbSession.RawIO.ReadString(50000);  
    }  
    else  
    {  
        throw new InvalidOperationException("Instrument is not connected.");  
    }  
}
```

Performs a reading from the instrument after a query command. It is used to obtain measurements or health responses from physical equipment.

```
public string Query(string command)  
{  
    if (mbSession != null)  
    {  
        Write(command);  
        return Read();  
    }  
    else  
    {  
        throw new InvalidOperationException("Instrument is not connected.");  
    }  
}
```

It executes a full query: it sends an SCPI command and reads its response. This method simplifies interaction with functions such as "?: *IDN?, READ?, MEAS?

```
public string[] ListResources()
```

```
{  
    using (var resourceManager = new ResourceManager())  
    {  
        var resources = resourceManager.Find("?*INSTR");  
        return resources.ToArray();  
    }  
}
```

Scans the system for instruments available through VISA. It allows you to automatically detect which computers are connected to your PC.

```
public string Identify()  
{  
    return Query("*IDN?");  
}
```

Returns the instrument ID using the standard *IDN? command. It is the first step to confirm correct communication and validate the equipment model.

```
public class Source : Instrumentos, I_Source  
{  
    public void Output_OFF()  
    {  
        Write("OUTP OFF");  
    }  
}
```

The Source class represents a programmable power supply controlled by SCPI. Allows you to turn the output on/off and adjust voltage/current precisely. It is designed for automatic testing where precise electrical control is required.

```
public class Multimeter : Instrumentos, I_Multimeter
{
    public string MeasureIdc(double range, double resolution)
    {
        Write($"CONF:CURR:DC {range},{resolution}");
        Write("SAMPLE:COUNT 30");
        return Query("READ?");
    }
}
```

The Multimeter class represents a programmable digital multimeter. It implements essential automated measurement functions: DC current, resistance, DC voltage, and continuity, all using SCPI commands. It is used in characterization routines, laboratory tests, and data logging.

Before developing the stored procedures, the creation and definition of the database was carried out, on which the structure of the project was based, which was provided by the master.

With the database previously analyzed and adapted to the requirements of the system, the stored procedures were developed, which were designed directly on this structure.

Also, it should be noted that, to test the functionality of the stored procedures, a C# program was created to execute the commands (*Query*) directly from there and extract information from the database and insert information into the database.

6.3- Development of stored procedures.

In this section, the most relevant SPs during the development of the project were discussed.

6.3.1- *AddTestPlanResult*.

These procedures form the group in charge of creating main elements within the system. *AddTestPlan* focuses on recording the results of a tested *TestPlan*, receiving parameters such as the number assigned to the test, the version among others in order to add them to the history.

AddTestPlanResult
<pre> USE [DB_reference222] GO SET ANSI_NULLS ON GO SET QUOTED_IDENTIFIER ON GO ALTER PROCEDURE [dbo].[SP_InsRes] @NS VARCHAR(50), @TP_ID INT, @res DECIMAL (18,6), @status VARCHAR (20), @DateTime DATETIME AS BEGIN SET NOCOUNT ON; INSERT INTO TestResult (NSerie, TestPlanId, TestResultValue, StatusCode, TestDateTime) VALUES (@@NS, @TP_ID, @res, @status, @DateTime); END </pre>

6.3.2- *Get_NextTRID*.

Get_NextTRID is responsible for getting the next *available TestResultId* in the *TestResult* table, ensuring that each test result has a unique ID, they share the same

logic based on reading the existing maximum value and generating the next consecutive number.

Get_NextTRID
<pre> ALTER PROCEDURE [dbo]. [Get_NextTRID] -- Add the parameters for the stored procedure here AS BEGIN -- SET NOCOUNT ON added to prevent extra result sets from -- interfering with SELECT statements. SET NOCOUNT ON; -- Insert statements for procedure here SELECT MAX(TR. TestResultId)+1 AS NEXT FROM TestResult AS TR END </pre>

6.3.3- *Get_NS_Repeated.*

Its function is to determine whether a serial number already exists within the Products table. To do this, it performs a direct search and returns a categorical result: "REPEATED" when it finds matches and "NOT REPEATED" when there are no previous records, to prevent duplicate products from being registered and to ensure that each serial number represents a single device within the process.

```

ALTER PROCEDURE [dbo]. [Get_NS_Repeated]
-- Add the parameters for the stored procedure here
@NSerie nvarchar (100)
AS
BEGIN
-- SET NOCOUNT ON added to prevent extra result sets from
-- interfering with SELECT statements.
SET NOCOUNT ON;

```

```
-- Insert statements for procedure here
if EXISTS (select 1 from Productos where NSerie = @NSerie)
begin
    select 'REPEATED' as
Status; End
else
begin
    select 'NO REPEATED' as Status;
End
END
```

6.3.4- *Report_TestPlan.*

Its function is to join the information from tables such as TestPlan, TestResult, TestStatus, and Products to see the behavior of the product within the testing process.

```
ALTER PROCEDURE [dbo]. [Report_TestPlan]
-- Add the parameters for the stored procedure here
AS
BEGIN
-- SET NOCOUNT ON added to prevent extra result sets from
-- interfering with SELECT statements.
SET NOCOUNT ON;

-- Insert statements for procedure here
select HCMC TestPlanId, HCMC TestName, HCMC TestNumber, HCMC Version, HCMC LowLimit, HCMC HighLimit,
HCMC MeasureUnit, TR. NSerie, P. TestResultValue, TR. TestDateTime, TR. StatusCode,
TS. Description as StatusDescription from TestPlan as TP
left join TestResult as TR on TP. TestPlanId = TR.
TestPlanId
```

```
order by TP. TestPlanId, TR. TestDateTime
END
```

6.3.5. *SP_Filter*.

This is responsible for being able to filter from the same parameters that were shown in *Fig.*

5.2.5. to get only the desired items and then pass them to the *Forms App* program.

Originally it was planned to perform 4 different stored procedures, but after some research it was possible to perform it within a single one, simplifying the amount of code to be used in both SQL and C# (since for each SP a method is required for the *productRepository*).

```
ALTER PROCEDURE [dbo].[SP_Filter]

    @OperationNumber int,
    @NS VARCHAR(50),
    @स_डेट Date,
    @एंड_डेट Date,
    @स्टैट Verture(20),
    @मॉडल Verture(10)

AS
BEGIN
    ----- Serial Number Lookup
    if @OperationNumber = 1
    Begin
        Select * From TestResult as r
        where r.NSerie like '%' + @NS + '%'

        End

    ----- Search by model
    if @OperationNumber = 2
    Begin
```

```
if @model = 'A'
    begin
        SELECT * FROM TestResult as r
        where r.TestPlanId >= 1
        And r.TestPlanId <=4
    End
if @model = 'B'
    Begin
        SELECT * FROM TestResult as r
        where r.TestPlanId >= 5
        And r.TestPlanId <=8
    End
End

----- Date search
iF @OperationNumber = 3
Begin
    SELECT * FROM TestResult as r
    where r.TestDateTime >= @S_Date
    And r.TestDateTime < DATEADD(Day, 1, @End_Date) --
End

-- Check for different status values if
@OperationNumber = 4
Begin
    Select * From TestResult
    Where StatusCode = @Stat
End
END
```

6.3.6- *Update_Producto_StatusCode.*

This group integrates all the procedures in charge of making modifications to existing records. UpdateModel allows you to adjust information associated with a registered model, such as its name or the assigned TestPlanId. Update_Producto_StatusCode updates the status of a specific product using its serial number, reflecting whether it is approved or rejected, finally, UpdateTestPlan allows you to modify parameters of an existing test plan, such as the test number, description, limits or unit, share the purpose of keeping the data updated without the need to delete or replicate records.

Update_Producto_StatusCode
<pre> ALTER PROCEDURE [dbo].[Update_Producto_StatusCode] -- Add the parameters for the stored procedure here @NSerie VARCHAR(100), @NuevoStatus VARCHAR(20) AS BEGIN -- SET NOCOUNT ON added to prevent extra result sets from -- interfering with SELECT statements. SET NOCOUNT ON; -- Insert statements for procedure here update Productos set TestStatusCode = @NuevoStatus where NSerie = @NSerie; END </pre>

6.4.- Development of the system.

As could be seen in the operation diagrams, the program can be explained in three fundamental operations of the main program, which will make use of the *service* controller to perform tasks that require the use of different classes. This approach was consistently applied across the various modules of the system, particularly in the classes responsible for communication with the *SeaLevel* device and in the validation of the stored procedures in the database.

6.4.1- Connection to the instrument.

When pressing the button to connect the instruments, you must first make sure that each one contains its respective port address. This is important because, if the trader makes a mistake when designating the direction of the instruments and they turn out to be wrong when connecting, they will not be able to make the proper trades. Therefore, a system was developed that at the time of making the connection looks for the identity of the instrument (from the IDN command). Since you are working with a source and a multimeter, you can look up the model of each one and thus make sure that they are the correct address. If they are not, the system immediately disconnects the two.

It is important to consider that this system was designed in mind with two completely different instruments, if they were the same, it would have to be compared in a different way to recognize what the respective direction of each should be.

In this same section, it also requires the connection of the *SeaLevel* for communication with the relays, this is given that as previously mentioned, these are the ones that will allow controlling which test will be carried out for the project. As this is recognized using the previously mentioned *System.IO.Ports* library, there are no problems when finding the address (it is also facilitated since it is called COM6 and is the only one of its kind).

In the case where the program is initialized and no addresses are found to assign an instrument or to the *SeaLevel*, it is possible to refresh them again using the *Refresh Locations* button that allows the user to load the port addresses again, but to avoid an error when connecting the instruments, all items in the *ComboBox* are *deselected*.

6.4.2- Product testing.

As the test station must make different measurements for each model, it is first important that when you press the "Initiate Test" button, it will proceed to select all the products that have not been measured. This is important because a queue will be generated that indicates the number of tests to be performed.

The first thing to consider is that these tests must show their progress within the interface, in the *Test Panel*, in order to comply with this requirement a thread was used that allows us to see how it will operate between each test, making sure to enable its *"isBackground"* property to be able to interact with the *TabControl windows*, but it is important to know that elements such as *Labels* or *LEDS* of the program cannot be refreshed within the *loop*, there are even some operations that modify other elements that are intolerant to the threads that interrupt the program. This represented an inconvenience during the development, since it prevented meeting the objectives of the project, the solution to this was to use an *invoke*, this becomes essential for the development of this program. Below you can see a fragment that uses *invoke*.

```
this. Invoke((MethodInvoker)() =>
{ R_Success.Value = true; meter2. Value = result; LB_R_MEAS.
  Text = "Voltage: " + result. ToString() + " V";
});
```

It is important to recognize that this should only be used when you want to update such elements and not encapsulate the entire thread operation, if this is done it generates a problem in the



Which is updated part of the elements in certain cases is not updated until the *Invoke* is run again or simply only partially visible the change. An example to understand better would be to suppose that we update a *label* to indicate that the voltage result of a test is 12.1945V, but at the time it is executed only the first tenth is partially observed.

Importantly, the thread is always executed, but to prevent the thread from taking readings without operator approval, a flag has been implemented that checks if the button was pressed to test the next product in the queue. Each product has to perform 4 tests until all are successful or one flags a failure, if the last one happens, all other tests for that product will be canceled and noted with an NT (*Not Tested*) but the one that failed will be marked with a "FAIL". In the case of a failure, it is easy to program, since for each *TestPlan* element, a *TestPlan* Result *must be created* that will be recorded in the report and the database. But if it fails, cases have to be made where depending on which part of the test failed, the program must record the other tests that will not be tested and mark them as previously mentioned with an NT which. An example is that if you fail the continuity test, the *TestPlan* must record the next 3 *TestPlans* as untested. The following excerpt shows how it is done.

case 3:

```
for (int i = 0; i < 3; i++)  
{ reg(ns, i + 1, tpID + i + 1); }  
NetTP_RId += 4;  
x = 4;  
break;
```

Where the cycle registers the necessary NT times and uploads it to the database (the *reg* method is responsible for registering these elements to the report and database).

But in order to evaluate whether or not it passed the tests, it is required to use the instrument classes, specifically the multimeter and source to be able to design the tests to be performed for the circuit and analyze if they comply with the limits of the *Test Plan*, depending on this, its result (i.e. the *Test Result*) will change the value of its status.

It is also important to consider that there can only be one single product, which allows us to distinguish them are the serial number contained in each of these. If a product is inserted into the queue, it must first be recognized if that product is not within the *TestResult* table, that is, the system analyzes whether it passed a test or not. If it turns out that model has already been tested, it cannot be queued for testing.

6.4.3- Search.

The interaction between the application in C# and the SQL Server database was implemented using the *ProductRepository* class, which is responsible for managing data access. This class executes read and write operations through stored procedures, ensuring structured, secure, and efficient communication with the database. To obtain the reports, the *DataTable* object was used, which allows the information to be transported and later uploaded to the DGV (*DataGridView*) where the information is to be presented.

To filter the different results of the database, a *ComboBox* is required to decide based on which parameter you want to filter (serial number, date range, model, status), where the interface will allow you to modify the values. The report can only be displayed when the search button is pressed, which from the stored procedure of *SP_Filter* will find and return the information found.



To be able to perform this filter on a single SP, an identification is required for SQL to recognize that it must be filtered, for this a variable called @OperationNumber was used, the following code snippet shows the value that must be met for serial numbers to be filtered.

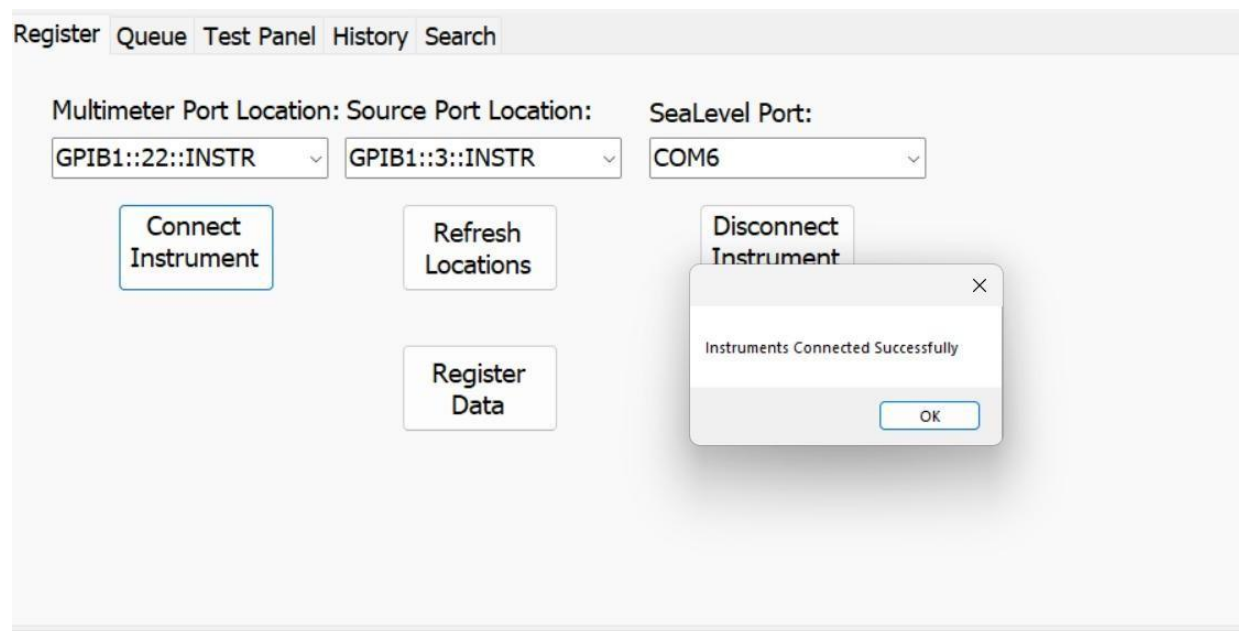
```
----- Serial Number Lookup  
if @OperationNumber = 1  
Begin  
Select * From TestResult as r  
where r.NSerie like '%' + @NS + '%'  
End
```

As you can see for the serial number, the percentage commands (or wildcards) will allow you to search for any serial number that contains the @NS values. Depending on what other data you want to filter the value of @OperationNumber will change.

7.- Limitations.

- The system will be limited to an on-premises environment, with no connection to external databases or cloud storage.
- The system depends on the instruments used being compatible with VISA and SCPI.
- The system does not include remote communication or distributed test execution.
- The queue will be handled in memory, so the information will be lost when you close the application.

8.-Results



The screenshot displays the 'Register' tab of the application. It features three dropdown menus for port configuration: 'Multimeter Port Location' (set to GPIB1::22::INSTR), 'Source Port Location' (set to GPIB1::3::INSTR), and 'SeaLevel Port' (set to COM6). Below these are four buttons: 'Connect Instrument', 'Refresh Locations', 'Disconnect Instrument', and 'Register Data'. A modal dialog box is open in the foreground, displaying the message 'Instruments Connected Successfully' with an 'OK' button.

Fig. 8.1 - Register results

Figure 8.1 shows the graphical interface of the developed system, corresponding to the instrument connection and configuration stage. In this phase, the software correctly detected the available resources via the VISA interface, identifying the digital multimeter and power supply connected via the GPIB bus, as well as the COM port assigned to the SeaLevel control module.

Register Queue Test Panel History Search					
ID TP	Test Number	Version	Test Name	LI	LS
5	101	1	Continuity Test	0	10
6	102	1	Voltage Test	11	12.5
7	103	1	CCW Test	-11.75	-10.7
8	104	1	Motor Vel Test	3.8125	4.56
9	201	2	Continuity Test	0	2
10	202	2	Voltage Test	5.635	6.46
11	203	2	CCW Test	-8.75	-7.75
12	204	2	Motor Vel Test	10.325	11.4
13	201	2	Continuity Test	0	2

Fig. 8.2- Queue results.

Figure 8.2 shows the Test Panel section of the developed system, which shows the list of Test Plans that have been successfully loaded into the execution queue. Each record corresponds to a specific test associated with a unique identifier (TP ID), a test number (Test Number) and a plan version (Version), which allows a structured and orderly control of the tests to be performed.

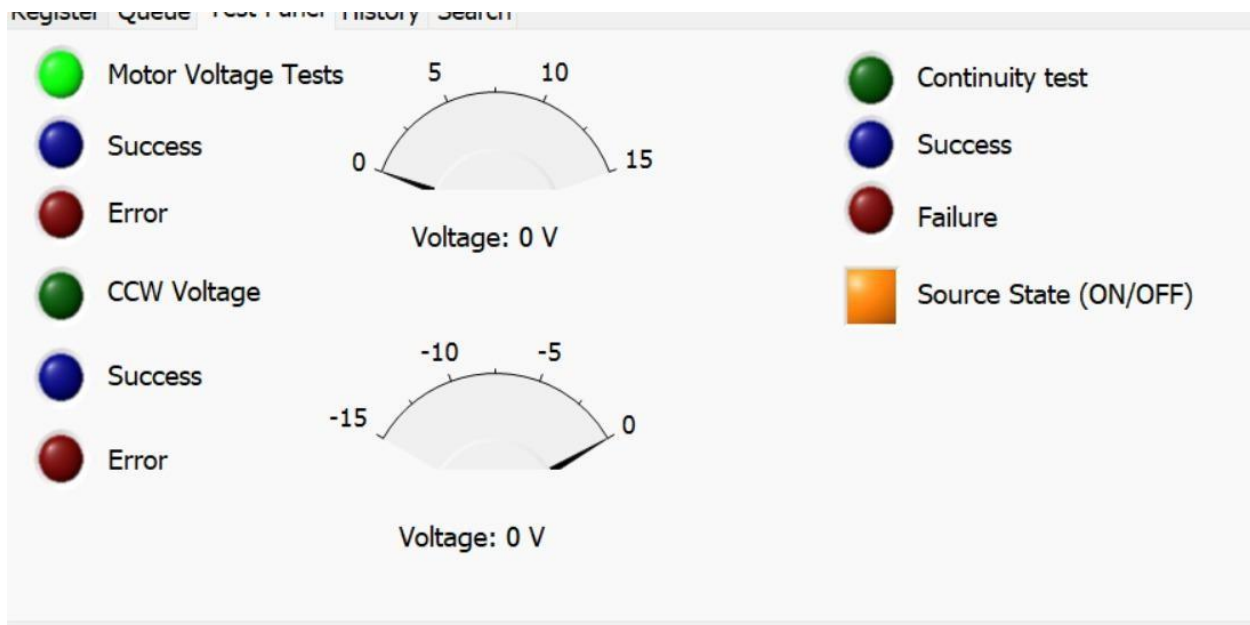


Fig. 8.3- Results of the test plans.

Figure 8.3 shows the system's Test Panel, which sees in real time the status of the electrical tests executed on the device under test. The interface integrates indicator lights and analog meters that facilitate the immediate interpretation of the system's behavior during each measurement stage.

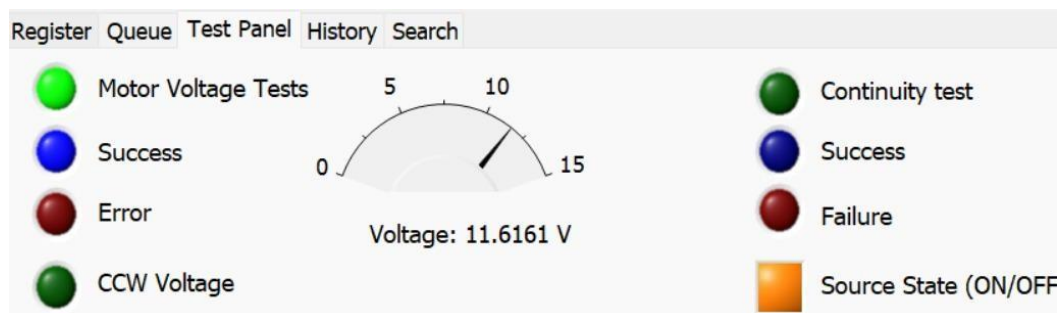


Fig. 8.4- Results of the test plans.

For the Motor Voltage Tests, the system features a meter that represents the voltage level applied to the motor, as well as visual status indicators that signal the test result. The green color indicates the activation or execution of the test, while the Success and Error indicators allow you to clearly identify if the measured value is within the established limits.

	Test Result Id	Series Number	Test Plan Id	Test Result Value	Status Code	Test Date Time
▶	1	SN0001A	1	2.9938	Passed	12/15/2...
	2	SN0001A	2	11.4712	Passed	12/15/2...
	3	SN0001A	3	-10.4365	Failed	12/15/2...
	4	SN0001A	4	0	NT	12/15/2...
*						

Fig. 8.5- Results of the test plans.

The interface corresponding to the system test results section is displayed. In this section, a table of results is displayed, in which those obtained after the execution of the test plans are recorded in a structured way.

The table includes fields such as *Test Result ID*, *Numbre Series*, *Test Plan ID*, *Test Result Value*, *Status Code*, and *Test Date Time*. These fields allow you to identify each test result, associate it with a number obtained, the status of the test and the date and time it was performed.

The results show different statuses, such as *Passed*, *Failed*, and NT (*Not Tested*), which indicates pending or untested tests. The numerical values recorded in the *Test Result Value* column corresponding to the measurements obtained during the tests.

The Display Test Plans *button remains at the top*, allowing you to update or view associated test plans, while the bottom is presented with a status bar that displays general system information, such as test queue status and system time.

Register Queue Test Panel History Search

Search Parameter

Serial Number Model Serial Number Status Code Dates

Search

Wednesday, December 17, 2025

Wednesday, December 17, 2025

	TestResult	NSerie	TestPlanId	TestResult\	StatusCode	TestDateTi
▶	9	SN9999A	1	1.000000	PASS	12/16/2...
	10	SN9999A	2	5.800000	FAIL	12/16/2...
	11	SN9999A	3	0.000000	NT	12/16/2...
	12	SN9999A	4	0.000000	NT	12/16/2...
*						

Fig. 8.6- Search filtered based on serial number.

Search Parameter

Status Model Serial Number Status Code Dates

	TestResult\	NSerie	TestPlanId	TestResult\	StatusCode	TestDateTi
▶	2	SN0001A	2	5.800000	FAIL	12/16/2...
	6	SN0003A	2	5.800000	FAIL	12/16/2...
	10	SN9999A	2	5.800000	FAIL	12/16/2...
•						

Fig. 8.7- Search filtered based on status.

The filter system is efficient as it allows the required test results to be easily located. Thanks to this mechanism, the operator can consult the information corresponding to different scenarios quickly and easily, optimizing data access and analysis.

9.- Conclusion.

The project meets most of the objectives set, since it allows different models to be tested and analyzed, determining whether or not they meet the necessary requirements to pass the test. This project represents the culmination of the knowledge acquired during the semester, applied to the development of a testing process so that it does not require the interaction of the operator and avoids possible damage to the board during the development of the tests.

This is important for our training as engineers, as it provides the knowledge for the development of skills and critical thinking for the design, implementation and validation of test systems.

10- Bibliographies

Braga, N. C. (n.d.). *Speed Control for DC Motors (ART139S)*. INCB.

[https://newtoncbraga.com.mx/articulos/9-articulos-tecnicos-y-proyectos/823-control-de-Speed-for-DC-ART139S motors](https://newtoncbraga.com.mx/articulos/9-articulos-tecnicos-y-proyectos/823-control-de-Speed-for-DC-ART139S-motors)

Securing Modbus Communications: Modbus TLS and Beyond. (s. f.). Sealevel Systems, Inc.

<https://www.sealevel.com/securing-modbus-communications-modbus-tls-and-beyond>

Test and measurement systems, a division of Emerson. (2022, September 2). NI.

https://www.ni.com/es.html?srsltid=AfmBOorkouy-59Py6O9v_8fvMiLDOXXD_vmeLT55Z9aV6c9zUeqN3E5z



Steven. (n.d.). *reversal of rotation in motors*. Scribd.

<https://www.scribd.com/document/793645332/inversion-de-giro-en-motores>

General Information about NI-VISA. (2006, August 31). NI.

<https://www.ni.com/es/support/documentation/supplemental/06/ni-virtual-instrument-software-architecture.html?srsId=AfmBOoq5-FRQ4VF7qsBVfyfRSvicLTbHRe-QCvkQjC5-ZqH-g8IhAWII>